

Revision Question Alternative Set 3

Question 1

- a) Write a query to calculate the total revenue generated by each product, partitioned by its category.
 - Use the SUM() window function to calculate the total price of products within each category.
 - The output should include product_id, product_name, category, and the category's total price (as category_total_revenue).

- b) Write a query to find the dates and IDs of all transactions that have a total_amount higher than the average amount of all transactions combined.
 - Use a subquery to calculate the average total_amount from the transactions table.
 - The output should include transaction_id, transaction_date, and total_amount

- c) Write a query to list every item sold in the transaction_items table along with its corresponding product name.
 - Use a JOIN to connect transaction_items with the products table.
 - The output should include transaction_id, product_name, and quantity.

- d) Write a query to categorize customers based on their location.
 - Use a CASE statement to create a new column called region_category:
 - 'Local' for customers in 'San Diego' or 'San Antonio'.
 - 'Coastal' for customers in 'Chicago' or 'Boston'.
 - 'Central' for customers in 'Austin' or 'Dallas', else 'Other'.
 - The output should include first_name, city, and region_category.

- e) Write a query to calculate a running total of stock_quantity across the products table.
- Use the SUM() window function to calculate the running total.
 - The running total should be ordered by product_id.
 - The output should include product_id, product_name, stock_quantity, and the running total (as running_stock_count).

Question 2

- a) Write a query to calculate the average transaction amount spent by customers, partitioned by the customer's city.
- Use the AVG() window function to calculate the average total_amount for each city.
 - Join the transactions and customers tables.
 - The output should include transaction_id, city, total_amount, and the city average (as city_avg_spend).
- b) Write a query to find the names of all products that have never been sold (i.e., they do not appear in the transaction_items table).
- Use a subquery to identify products whose product_id is not present in the transaction_items table.
 - The output should include product_name.
- c) Write a query to list all transactions including the email of the customer who made them.
- Use a JOIN to connect the transactions table with the customers table.
 - The output should include transaction_id, transaction_date, and email.

d) Write a query to categorize products based on their price.

- Use a CASE statement to categorize products as:
- 'Premium' if price is greater than 10.00.
- 'Mid-Range' if price is between 5.00 and 10.00.
- 'Budget' if price is below 5.00, else 'Unpriced'.
- The output should include product_name, price, and price_tier

e) Write a query to find the minimum and maximum stock quantity available within each product category.

- Use the MIN() and MAX() functions to find these values.
- Group the results by category.
- The output should include category, minimum_stock, and maximum_stock.